Introduction

If you've spent any time preparing for coding interviews, you've probably noticed that certain problems keep showing up on every "hard" list: LRU Cache, Trapping Rain Water, and near the top of that list — Median of Two Sorted Arrays. It has a reputation for being intimidating, and honestly, that reputation is earned. Not because the code is long, but because the *insight* required to get to an optimal solution isn't obvious at all.

This problem is a favorite at companies that care about algorithmic depth, because it tests something more specific than "can you code a merge." It tests whether you can reason about a search space that isn't made of numbers, but of *positions* — whether you can recognize that a partition problem is secretly a binary search problem in disguise. That skill generalizes far beyond this one LeetCode question. It shows up in database engines computing percentiles across sharded, sorted data, in distributed log systems merging sorted streams without materializing the whole thing, and in any system where "sorted but split across sources" is the norm rather than the exception.

Learning this problem properly means learning to ask a different question than the one you're handed. Instead of "how do I merge these," you learn to ask "where would the merge point be, without doing the work of merging." That reframing is the whole game, and once you see it, you can't unsee it.

Problem Overview

You're given two sorted arrays, `nums1` with `m` elements and `nums2` with `n` elements. If you were to merge them into a single sorted array, you need to find the median of that merged array — without actually building it.

The median is defined the usual way:

- If the total number of elements is odd, the median is the middle element.
- If the total number of elements is even, the median is the average of the two middle elements.

The constraint that makes this problem interesting is the time complexity requirement: the overall algorithm must run in `O(log(m + n))` time. That single requirement rules out any approach that touches every element, which eliminates the two most "natural" solutions before you even start coding.

Example

Example 1

```
nums1 = [1, 3]
nums2 = [2]
```

Merged: `[1, 2, 3]` → odd length (3), so the median is the middle element: **2**.

Example 2

```
nums1 = [1, 3, 5]
nums2 = [2, 4, 6, 8]
```

Merged: `[1, 2, 3, 4, 5, 6, 8]` → 7 total elements (odd), middle index is 3 (0-indexed), which holds **4**.

Example 3 (even total)

```
nums1 = [1, 2]
nums2 = [3, 4]
```

Merged: `[1, 2, 3, 4]` → 4 elements (even), median is the average of index 1 and index 2: `(2 + 3) / 2 = 2.5`.

Intuition

Start with the obvious move: merge both arrays, sort the result, and grab the middle. That works, but sorting the merged array costs `O((m+n) log(m+n))`, and even a simple linear merge (since both inputs are already sorted) costs `O(m+n)`. Neither hits the logarithmic requirement — so the problem is explicitly telling you: stop thinking about *values*, start thinking about *positions*.

Here's the reframing that unlocks the solution. The median doesn't care about every element — it only cares about how many elements are smaller than it and how many are larger. So instead of asking "what is the median value," ask a different question: **"If I could draw one imaginary line through each array, splitting each into a left half and a right half, where would those two lines need to sit so that combining both left halves gives you exactly the correct number of the smallest elements?"**

That's a partition. If you get the partition right — meaning every element left of both cuts is smaller than every element right of both cuts — the median falls directly out of the four values sitting next to the cuts, without ever touching anything else in the arrays.

Now, how do you *find* that correct partition quickly? This is where binary search comes in. Picking a cut position in one array automatically determines the cut position in the other array, because the total count on the left side is fixed (it has to equal half the combined length). So you only need to binary search over cut positions in *one* array. And to keep the search space as small as possible — which is the whole point of hitting `log` time — you binary search whichever array is **shorter**. Search the longer one and your valid range of cut positions can be badly skewed or even invalid; search the shorter one and the range is always well-behaved.

At each guess, you check four border values — the element just left of each cut and just right of each cut. If both "left" values are less than or equal to both "right" values, you've found the correct partition and the median is right there. If not, you nudge your guess left or right, exactly like a normal binary search, because you can tell from *which side* is violating the condition whether your cut in the shorter array was too far right or too far left.

Brute Force Solution

Idea: Merge both sorted arrays into one sorted array (using a simple linear merge since they're both already sorted), then index into the middle.

Algorithm: 1. Use two pointers to walk both arrays simultaneously, always taking the smaller current element. 2. Append to a result list until both arrays are exhausted. 3. Index into the middle (or average the two middle elements if the length is even).

Advantages: Dead simple to write and reason about. Very hard to get wrong.

Disadvantages: Uses `O(m+n)` extra space for the merged array, and even with a smart linear merge, it does `O(m+n)` work — nowhere near the `O(log(m+n))` the problem demands.

Complexity: Time `O(m+n)`, Space `O(m+n)`.

Optimal Solution

The optimal approach binary searches over cut positions in the **shorter** array. Call the shorter array `A` (length `m`) and the longer one `B` (length `n`).

We want to choose a cut `i` in `A` (0 to `m` elements on A's left side) and a cut `j` in `B` such that:

```
i + j = (m + n + 1) // 2
```

This guarantees the combined left side always has the correct count regardless of whether `m+n` is odd or even. Since `i` determines `j` directly (`j = half - i`), we only need to binary search `i`.

For a given `i` and `j`, define the four border values:

Value	Meaning
`Aleft`	`A[i-1]` if `i > 0`, else `-infinity`
`Aright`	`A[i]` if `i < m`, else `+infinity`
`Bleft`	`B[j-1]` if `j > 0`, else `-infinity`
`Bright`	`B[j]` if `j < n`, else `+infinity`

Using infinity for missing borders means you never need a special case for an empty array or a cut sitting at the very edge — the comparisons just work.

The cut is correct when:

```
Aleft <= Bright  and  Bleft <= Aright
```

- If `Aleft > Bright`, your cut in `A` is too far right — move `i` left.
- If `Bleft > Aright`, your cut in `A` is too far left — move `i` right.

Once the condition holds: - Odd total length → median is `max(Aleft, Bleft)`. - Even total length → median is `(max(Aleft, Bleft) + min(Aright, Bright)) / 2`.

Diagram

```
        A cut (i)               B cut (j)
A: [ ... Aleft | Aright ... ]
B: [ ... Bleft | Bright ... ]

Combined left side  = i + j elements  (always the smaller/equal half)
Combined right side = (m-i) + (n-j) elements
```

Python Code

```
from typing import List


def find_median_sorted_arrays(nums1: List[int], nums2: List[int]) -> float:
    # Always binary search the shorter array to keep the range valid and small.
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
```

```
    half = (m + n + 1) // 2

    lo, hi = 0, m
    while lo <= hi:
        i = (lo + hi) // 2   # cut position in the shorter array
        j = half - i         # forced cut position in the longer array

        a_left = nums1[i - 1] if i > 0 else float("-inf")
        a_right = nums1[i] if i < m else float("inf")
        b_left = nums2[j - 1] if j > 0 else float("-inf")
        b_right = nums2[j] if j < n else float("inf")

        if a_left <= b_right and b_left <= a_right:
            if (m + n) % 2 == 1:
                return max(a_left, b_left)
            return (max(a_left, b_left) + min(a_right, b_right)) / 2
        elif a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1

    raise ValueError("Input arrays must be sorted")
```

Code Walkthrough

- **Swap step:** `if len(nums1) > len(nums2): nums1, nums2 = nums2, nums1` guarantees we always binary search the shorter array — this is what keeps the search range bounded correctly.
- **`half`:** the number of elements that must sit on the left side of the combined partition. Using `(m + n + 1) // 2` handles both even and odd totals with the same formula.
- **`lo, hi = 0, m`:** the cut in the shorter array can range from 0 (nothing on the left) to `m` (everything on the left).
- **`i` and `j`:** `i` is our current guess; `j` is forced by the constraint that the two cuts together must produce exactly `half` elements on the left.
- **Border values with infinity:** eliminates every special case for empty arrays or edge cuts — comparisons against infinity behave exactly as you'd want.
- **The correctness check:** if both cross-comparisons hold, the partition is valid and the median can be read directly.
- **Adjustment branches:** mirror standard binary search — narrow the range based on which side violated the condition.

Dry Run

`nums1 = [1, 3, 5]`, `nums2 = [2, 4, 6, 8]`

`len(nums1) = 3 <= len(nums2) = 4`, so no swap. `m=3, n=4`, `half = (3+4+1)//2 = 4`.

`lo=0, hi=3`

Iteration 1: `i = 1`, `j = 3` `a_left = nums1[0] = 1`, `a_right = nums1[1] = 3` `b_left = nums2[2] = 6`, `b_right = nums2[3] = 8` Check: `a_left(1) <= b_right(8)` ✓, but `b_left(6) <= a_right(3)` ✗ → move right: `lo = 2`

Iteration 2: `i = 2`, `j = 2` `a_left = nums1[1] = 3`, `a_right = nums1[2] = 5` `b_left = nums2[1] = 4`, `b_right = nums2[2] = 6` Check: `a_left(3) <= b_right(6)` ✓, `b_left(4) <= a_right(5)` ✓ → valid partition!

Total length `m+n = 7` is odd → median = `max(a_left, b_left) = max(3, 4) = 4`.

Matches the merged array `[1,2,3,4,5,6,8]`, where the middle element is `4`. ✅

Complexity Analysis

- **Time:** `O(log(min(m, n)))` — the binary search only runs over the shorter array's index range, and each iteration does constant work.
- **Space:** `O(1)` — only a handful of scalar variables regardless of input size.

This is correct because binary search halves the candidate range each iteration, and the range size is bounded by the length of the shorter array, not the combined length.

Alternative Solutions

Two-pointer merge (without full materialization): walk both arrays with two pointers, counting elements until you reach the middle position(s), tracking just the last one or two values you'd have appended instead of building the whole merged array.

- Time: `O(m+n)`
- Space: `O(1)`
- Simpler to write and debug than the partition approach, and in most real production code, `m+n` isn't large enough for the difference between linear and logarithmic to matter. It's a very reasonable engineering tradeoff outside of an interview setting — just not what this specific problem is grading you on.

Edge Cases

- **One array is empty:** the infinity-based border logic handles this automatically — no special branch needed.
- **Arrays of drastically different sizes:** always binary searching the shorter array keeps the search space valid no matter how skewed the sizes are.
- **All elements in one array smaller than all elements in the other:** the cut in the shorter array will land at one extreme (0 or `m`), and infinity borders handle it cleanly.
- **Duplicate values across arrays:** the algorithm only compares values relative to partition position, so duplicates don't break anything.
- **Single-element arrays:** works the same way; `m` or `n` being 1 just narrows the binary search range immediately.

Common Mistakes

1. **Binary searching the longer array instead of the shorter one** — this can produce an invalid or unnecessarily large search range and breaks the log-time guarantee.
2. **Forgetting to swap when `nums1` is longer than `nums2`** — leads to index errors or incorrect results on skewed inputs.
3. **Off-by-one errors in the `half` formula** — using `(m+n)//2` instead of `(m+n+1)//2` breaks the odd-length case.
4. **Not handling edge cuts with infinity** — writing manual `if i == 0` / `if i == m` branches instead of using `-inf` / `inf` sentinels, which is more error-prone.
5. **Confusing which condition means "move left" vs "move right"** — swapping `a_left > b_right` and `b_left > a_right` silently produces wrong answers instead of crashing.
6. **Returning an integer instead of a float for even-length medians** — dividing with `/` in Python 3 handles this correctly, but it's a common bug in other languages.

Interview Questions

- Why does binary searching the shorter array guarantee correctness and the log(min(m,n)) bound?
- How would you extend this to find the k-th smallest element across two sorted arrays instead of just the median?
- What happens to the algorithm if the arrays aren't sorted? Can you detect that case?
- How would this generalize to finding the median across more than two sorted arrays?
- Can you prove the binary search loop always terminates?

Similar Problems

- **Kth Smallest Element in a Sorted Matrix** — same partition/binary-search-on-answer mindset applied to a 2D structure.
- **Merge Sorted Array** — the brute-force building block this problem deliberately avoids using at scale.
- **Find K-th Smallest Pair Distance** — another problem where binary searching over "positions/values" beats brute-force enumeration.
- **Split Array Largest Sum** — binary search over a monotonic answer space, the same core technique in a different costume.

Key Takeaways

The real lesson here isn't the specific code — it's the pattern: when a problem asks for a value derived from combining sorted data, and demands better-than-linear time, stop thinking about merging and start thinking about **partitions**. Binary search doesn't only work on sorted arrays of numbers; it works on any monotonic search space, including "where should this cut go." Once that clicks, a whole category of hard problems becomes approachable.

Watch the Video

For the full walkthrough with diagrams, the brute-force build-up, and two live quizzes to test your understanding as you go, watch the video here: {LINK}

About the Series

This is part of the Daily Python LeetCode series, where one problem gets broken down from first principles every day — starting with the brute-force approach, building up the intuition, and arriving at the optimal solution the way you'd actually discover it in an interview, not just handing you memorized code.

Call To Action

If this breakdown helped the partition trick finally click, drop a like on the video and subscribe on YouTube for tomorrow's problem. Subscribe to the Substack for the full written breakdown of every solution, and leave a comment if you'd like to see the k-th smallest element generalization covered next. Sharing this with someone prepping for interviews helps more than you'd think.

Quick Review

Two sorted arrays, need median in log time — which pattern?

Binary search on the partition/cut point, not a merge. Trigger: 'sorted arrays' + 'log time' + median/kth element.

Time complexity of the partition-based median solution?

O(log(min(m,n))) — binary search runs on the shorter array only.

Space complexity of the cut-based approach?

O(1) — no merged array is built, just index pointers and four border values.

Trickiest edge case in this problem?

Empty array, or a cut landing at index 0/length (use -infinity/+infinity sentinels for out-of-range borders).

Naive merge approach vs the two-pointer cut trick?

Merge: O(m+n) time, O(m+n) or O(1) space but slow. Cut trick: O(log min(m,n)) time by binary searching partitions instead of merging.

Follow-up: why binary search the shorter array, not either?

Bounds the search space to O(log min(m,n)) and guarantees the complementary cut in the longer array stays within valid range.